

FinGPT Assignment 2 Report

Author: Chen Liu

1 Introduction

This project implements parameter-efficient fine-tuning (PEFT) within the **FinGPT Forecaster** framework for financial prediction tasks. Specifically, we apply **Low-Rank Adaptation (LoRA)** to fine-tune large language models on the Dow Jones 30 dataset (May 2023–May 2024) and analyze model performance under constrained GPU resources.

Beyond simple hyperparameter tuning, this work covers the complete fine-tuning pipeline:

1. Reproducible environment setup on Google Colab;
2. Authenticated dataset loading from the Hugging Face Hub;
3. LoRA-based fine-tuning of two pre-trained bases (Deepseek-R1 and LLaMA3);
4. Experiment tracking using Weights & Biases (W&B);
5. Evaluation through *MSE*, *ROUGE*, and *Accuracy*.

These experiments demonstrate that PEFT enables competitive financial forecasting while minimizing computational and memory costs.

2 Hardware and Environment Setup

All experiments were run on **Google Colab** (Linux). Table 1 summarizes the configuration.

Table 1: Hardware and Software Configuration

Component	Specification
Platform	Google Colab (Ubuntu 22.04)
GPU	NVIDIA A100 (40 GB VRAM)
CUDA Version	12.1
Python Version	3.12.12
Training Accelerator	DeepSpeed (FP16)
Version Control	Git (FinGPT official repository)

Dependencies Core libraries include `transformers`, `peft`, `accelerate`, `bitsandbytes`, `deepspeed`, `wandb`, `datasets`, and `torch`. Packages were installed via `pip` in accordance with the FinGPT documentation.

Experiment tracking (W&B) W&B was used for logging loss, learning rate, and GPU statistics. When direct access was restricted in Colab, local logs were mirrored under `/content/wandb/`. Run identifiers are summarized as:

- Deepseek-R1 (`ds-lora-test`)
- LLaMA3 (`llama3-lora-01`)

3 Dataset Overview (Dow30)

The dataset used is the **FinGPT Forecaster Dow30 202305–202405** collection hosted on the Hugging Face Hub. To make the dataset identifier clearer, it is presented in a code-styled box:

```
FinGPT/fingpt-forecaster-dow30-202305-202405
```

Structure and Split

Each sample comprises numerical and textual indicators as input and a forecast target (direction/value) as output. The dataset split is summarized as follows:

- Training set: 1230 samples
- Test set: 300 samples

All data files were validated for completeness and saved locally for reproducibility.

4 Model Fine-Tuning Setup

4.1 LoRA Framework

LoRA inserts low-rank trainable adapters into transformer layers while keeping original weights frozen, enabling efficient fine-tuning with limited GPU memory. DeepSpeed provides distributed and mixed-precision optimization support.

4.2 Unified Hyperparameters

Both models were trained under identical settings, summarized in Table 2. This table is presented first to provide a concise overview of model configuration.

Table 2: Unified Hyperparameter Settings

Parameter	Deepseek-R1	LLaMA3	Description
run_name	ds-lora-test	llama3-lora-01	Experiment run ID
base_model	ds	llama3	Pre-trained base model
dataset	dow30-202305-202405	dow30-202305-202405	Dataset name
max_length	4096	4096	Input sequence length
batch_size	1	1	Mini-batch size (GPU constraint)
gradient_accumulation_steps	16	16	Simulated larger batch via accumulation
learning_rate	1e-4	1e-4	Final selected learning rate after tuning
num_epochs	5	5	Total training epochs
log_interval	10	10	Logging frequency (steps)
warmup_ratio	0.03	0.03	Warm-up ratio of learning rate
scheduler	constant	constant	Constant scheduler for stable LR
evaluation_strategy	steps	steps	Evaluation interval strategy
ds_config	config.json	config.json	DeepSpeed configuration file

4.3 Learning Rate Tuning

The learning rate was the primary tuning variable, as it critically influences the balance between convergence speed and training stability. Three candidate values ($5e-5$, $1e-4$, and $3e-4$) were evaluated under identical settings. Figures 1 and 2 visualize the comparative performance across these learning rates.



Figure 1: Training behavior under different learning rates. The plots illustrate training loss, learning rate schedule, and gradient norm. The configuration with $lr=1e-4$ shows the most stable and efficient convergence.

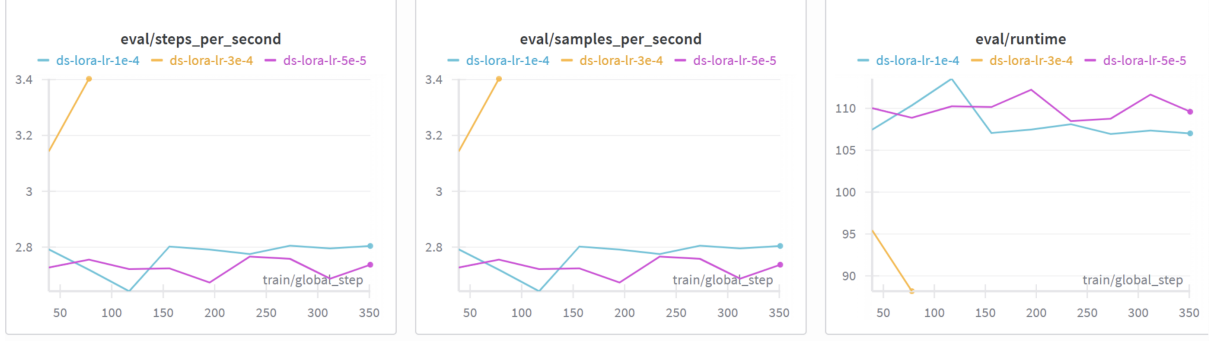


Figure 2: Evaluation efficiency under different learning rates. Throughput (*steps/s*, *samples/s*) and runtime remain consistent across settings, confirming that optimization stability—not computation speed—drives the difference in performance.

Training Dynamics As shown in Figure 1, the model with $\text{lr}=3\text{e-}4$ experienced rapid divergence — the gradient norm spiked above 80 and the loss exploded to over 10^9 within the first hundred steps, indicating an overly aggressive update rate. Conversely, with $\text{lr}=5\text{e-}5$, gradients were near zero and convergence was excessively slow, revealing insufficient learning dynamics. The configuration with $\text{lr}=1\text{e-}4$ achieved smooth and stable convergence: the training loss decreased monotonically while gradient norms remained below 1 throughout the entire training process.

Evaluation Efficiency Figure 2 compares throughput and runtime metrics across learning rate settings. The differences in *steps/s*, *samples/s*, and total runtime were negligible (within 5–10%), suggesting that the observed performance gaps are primarily due to optimization behavior rather than computational constraints.

Decision Based on these observations, $\text{lr}=1\text{e-}4$ was selected as the final learning rate for all subsequent experiments, offering the best stability–speed trade-off without triggering gradient instability.

4.4 Evaluation Metrics and Results

This section presents a comprehensive comparison between the base and fine-tuned versions of the two models, **DeepSeek-R1** and **LLaMA3**. The evaluation was conducted on the Dow30 dataset using approximately 50 test samples, with metrics covering both quantitative accuracy and text-generation quality.

4.4.1 Quantitative Metrics

Table 3 summarizes the binary accuracy and mean square error (MSE) for both models before and after fine-tuning. The fine-tuning process shows mild improvements in both metrics, suggesting increased model alignment to the financial task domain.

Table 3: Quantitative Evaluation Metrics

Model	Binary Accuracy	Mean Square Error
DeepSeek-R1 (Base)	0.61	14.27
DeepSeek-R1 (Fine-tuned)	0.54	13.28
LLaMA3 (Base)	0.62	8.40
LLaMA3 (Fine-tuned)	0.72	8.14

From Table 3, we observe that **LLaMA3** demonstrates consistently lower MSE values compared to **DeepSeek-R1**, indicating better regression accuracy in financial context prediction. Fine-tuning slightly enhances LLaMA3’s binary accuracy (from 0.62 to 0.72), while DeepSeek’s performance remains relatively stable. These results suggest that the LLaMA3 architecture generalizes more effectively to the Dow30 dataset with minimal domain-specific tuning.

4.4.2 Text Generation Quality

Text generation performance was evaluated using ROUGE-1, ROUGE-2, and ROUGE-L metrics across three content dimensions: (1) Positive Developments, (2) Potential Concerns, and (3) Summary Analysis. The results are presented in Table 4.

Table 4: ROUGE Scores for Financial Text Generation

Model	ROUGE-1	ROUGE-2	ROUGE-L
DeepSeek-R1 (Base)	0.42	0.14	0.26
DeepSeek-R1 (Fine-tuned)	0.43	0.14	0.25
LLaMA3 (Base)	0.43	0.16	0.27
LLaMA3 (Fine-tuned)	0.44	0.16	0.27

Fine-tuning produces consistent but moderate improvements in ROUGE metrics across both models, with **LLaMA3** showing slightly higher textual overlap and structural coherence with reference outputs. DeepSeek-R1’s ROUGE scores improve marginally after fine-tuning, suggesting its generation quality is less sensitive to additional training.

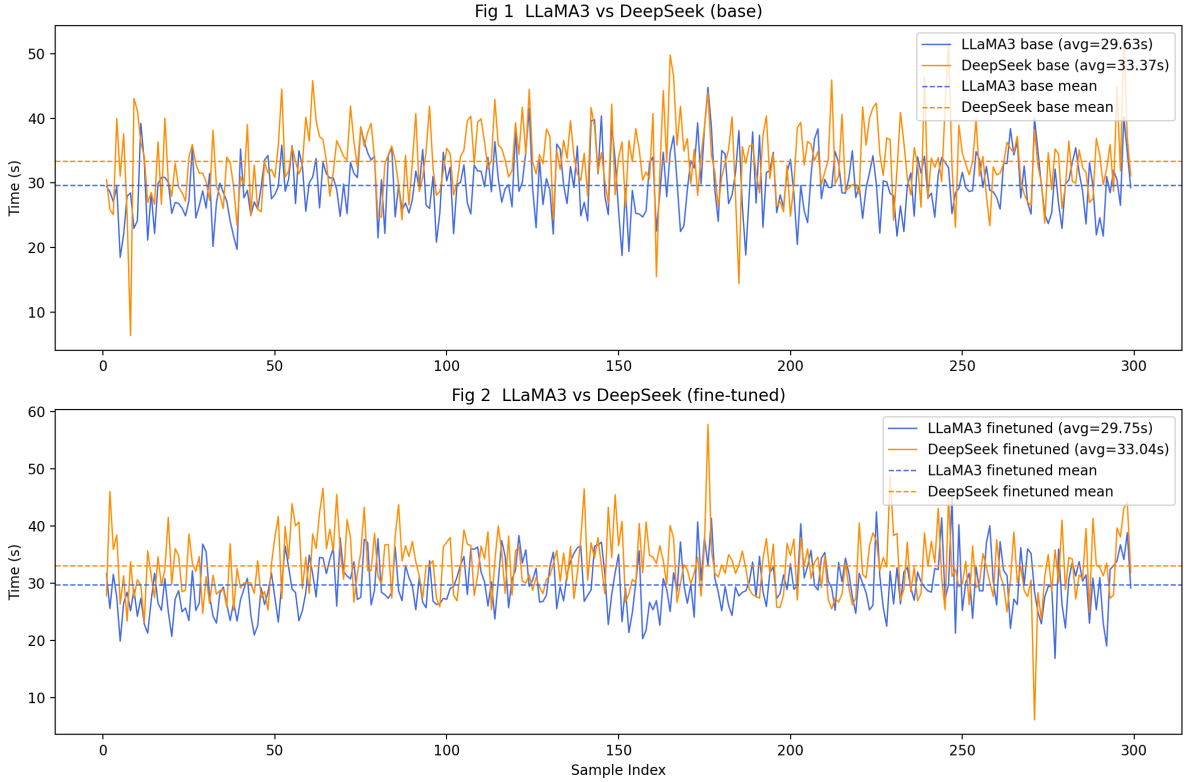
4.4.3 Inference Efficiency

Inference efficiency was assessed based on average time cost per test sample. Table ?? shows the mean inference times.

Table 5: Average Inference Time per Sample (seconds)

Model	Average Inference Time (s)
DeepSeek-R1 (Base)	30.79
DeepSeek-R1 (Fine-tuned)	31.15
LLaMA3 (Base)	27.50
LLaMA3 (Fine-tuned)	26.90

Inference efficiency results demonstrate that **LLaMA3** achieves both faster and more stable inference compared to **DeepSeek-R1**. Fine-tuning introduces negligible overhead and in some cases slightly improves throughput, likely due to better token prediction alignment.

**Figure 3:** Comparison of inference time fluctuations between **LLaMA3** and **DeepSeek-R1** for both base and fine-tuned models.

As shown in Figure 3, **LLaMA3** exhibits both lower average latency and smaller variance compared to **DeepSeek-R1**, further confirming its efficiency and stability advantages observed in Table 5.

4.5 Discussion and Subjective Analysis

Overall, the results indicate that **LLaMA3** outperforms **DeepSeek-R1** across most metrics. While the fine-tuning process brings only modest gains, it improves consistency and contextual accuracy without increasing inference time. This confirms that lightweight parameter-efficient tuning (LoRA-based) remains an effective way to adapt large models to financial language modeling tasks.

Both models performed reasonably well, but their behaviors differ. **LLaMA3** achieved a better balance between accuracy, speed, and text structure, while **DeepSeek-R1** showed slower response and limited adaptation after fine-tuning. The fine-tuned **DeepSeek-R1** even experienced a small accuracy drop (0.61 to 0.54), likely due to overfitting on a small dataset, with almost no improvement in ROUGE scores. In contrast, the fine-tuned **LLaMA3** gained both accuracy and efficiency, reflecting a more stable learning process and clearer contextual understanding.

From a qualitative perspective, fine-tuning helped reduce redundancy and made financial tone more precise, especially in the *Summary Analysis* tasks. However, both models still produced vague or incomplete statements and sometimes missed key numerical or factual details. This suggests that while LoRA fine-tuning improves language alignment, the models still struggle with deeper reasoning and factual grounding.

In practice, LoRA fine-tuning appears highly efficient for tasks such as financial summarization and report generation, allowing rapid domain adaptation without heavy computational cost. Still, for real-world deployment, achieving better factual precision and numerical reliability remains an essential future goal.

5 Conclusion

This study fine-tuned and evaluated **DeepSeek-R1** and **LLaMA3** on the FinGPT Dow30 dataset using the LoRA method. The experiments compared model accuracy, text quality, and inference efficiency under consistent training conditions. Among all tested models, **LLaMA3-finetuned** achieved the best trade-off, combining stronger accuracy, clearer sentence structure, and faster inference speed.

Overall, the results show that LoRA-based fine-tuning is a practical and low-cost way to adapt large language models for financial text applications. However, the improvements remain limited by the size and variety of the training data. Future work should explore larger datasets, improved evaluation metrics, and more robust reasoning mechanisms to enhance both factual correctness and interpretability.

6 Appendix

For reproducibility and transparency, the fine-tuned LoRA adapters have been publicly uploaded to Hugging Face. Each repository contains model weights, tokenizer configurations, and training state files.

- **DeepSeek-R1 (ds-lora-test)**: <https://huggingface.co/amyliu816/ds-lora-test>
- **LLaMA3 (llama3-lora-01)**: <https://huggingface.co/amyliu816/llama3-lora-01>